

Deadlock Prevention

1 Main Idea

Deadlock Prevention

Deadlock prevention means designing the system so that at least one of the four necessary deadlock conditions can never hold.

Deadlock avoidance is often impractical because it requires knowing future resource requests. Prevention avoids that problem by changing system rules structurally.

Prevention Rule

If any one Coffman condition is impossible, deadlock is impossible.

2 Four Conditions to Attack

Condition	Meaning
Mutual exclusion	A resource can be assigned to only one process at a time.
Hold and wait	A process can hold resources while requesting more.
No preemption	A granted resource cannot be forcibly taken away.
Circular wait	Processes form a cycle, each waiting for a resource held by the next.

3 Attacking Mutual Exclusion

Idea

If resources are never assigned exclusively, then deadlock cannot occur through those resources.

For some data, this is possible: read-only data can be shared by many processes at the same time. But many resources cannot be shared directly. Two processes writing to the same printer at the same time would produce mixed output.

Spooling

Spooling avoids direct printer sharing. Processes write output files to disk, and only the printer daemon uses the physical printer.

This removes deadlock involving the printer because the printer daemon does not request other resources. However, spooling can create another possible deadlock if disk spooling space fills up.

Spooling-space deadlock

If two processes each fill half the spooling space and both need more space to finish their output, neither can complete. The deadlock has moved from the printer to disk space.

4 Attacking Hold and Wait

Idea

Prevent processes from holding resources while waiting for more resources.

There are two common methods.

Method	Rule	Problem
Request all resources first	A process must request everything before it starts.	Processes may not know their future needs; resources are wasted.
Release before requesting	A process must release all held resources before asking for new ones.	May be inconvenient or impossible for some workflows.

Resource waste

If a process needs an input tape first, then computes for one hour, then needs an output tape and plotter, requesting everything at startup wastes the output tape and plotter during the computation hour.

Where it is used

Some mainframe batch systems require jobs to list all required resources before execution. This prevents deadlock but burdens the programmer and wastes resources.

5 Attacking No Preemption

Idea

If resources can be forcibly taken away safely, then deadlock can be broken or prevented.

This works only for resources whose state can be saved and restored safely. It is difficult for physical resources in the middle of an operation.

Resource	Preemption possible?	Reason
Memory pages	Often yes	Pages can be swapped out and restored.
Printer while printing	Usually no	Taking it away may corrupt output.
Database record lock	Usually no	The protected update must remain consistent.
Spooled printer output	More manageable	The real printer is used only by the daemon.

Virtualization limit

Spooling virtualizes the printer, but not all resources can be virtualized. Database records and operating-system tables still require locks.

6 Attacking Circular Wait

Idea

Prevent cycles by forcing all processes to request resources in a fixed global order.

One strict rule is: a process may hold only one resource at a time. If it needs another, it must release the first. This prevents circular wait, but it is often too restrictive.

A more practical method is to assign every resource type a number. Processes must request resources in increasing numerical order.

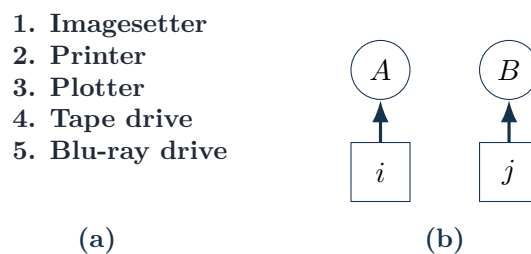


Figure: (a) Numerically ordered resources. (b) A resource graph.

Why ordering works

If every process requests resources only in increasing order, a circular wait cannot form. A cycle would require some process to request a lower-numbered resource after holding a higher-numbered one, which the rule forbids.

7 Proof Intuition for Resource Ordering

Assume every resource has a unique number. At any moment, look at the highest-numbered resource currently assigned. The process holding it cannot be waiting for any lower-numbered resource, because that is forbidden. It can only request higher-numbered resources, which are not already held by any process with a higher resource. Therefore, some process can eventually finish and release resources.

Practical difficulty

Finding one global order that works for all resources can be hard. Real systems may have many abstract resources: process-table slots, disk space, database records, locks, files, and devices.

8 Summary Table

Condition attacked	Prevention method	Main disadvantage
Mutual exclusion	Share resources where possible; use spooling for devices.	Many resources cannot be shared; spooling may shift deadlock to disk space.
Hold and wait	Require all resources at start, or release held resources before requesting more.	Wastes resources and requires advance knowledge.
No preemption	Take resources away when needed, or virtualize them.	Many resources cannot be safely preempted.
Circular wait	Number resources and require increasing request order.	A global ordering may be hard to design.